

Feature Selection

Learning objectives

- Why feature selection is required.
- Distinguish between feature selection and feature extraction.
- Identify irrelevant, redundant, and highly correlated features.
- Apply filter, wrapper, and embedded feature-selection methods.
- **Use Python and Scikit-learn for feature selection.**
- **Select an appropriate method for classification and regression problems.**
- **Avoid data leakage while performing feature selection.**

What is a feature?

- A **feature** is an input variable used by a machine-learning model to make predictions.
- For example, consider predicting a student's final marks.

Feature	Meaning
Study hours	Number of hours studied
Attendance	Attendance percentage
Previous marks	Marks obtained previously
Sleep hours	Average sleeping time
Final marks	Target variable

Here, the first four columns are features, and Final marks is the target

What is feature selection?

- **Feature selection** is the process of **selecting the most useful input features** and **removing** features that are **irrelevant, redundant, or noisy**.
- Suppose a dataset contains:

$$X = \{x_1, x_2, x_3, \dots, x_{20}\}$$

After feature selection, the model may use only:

$$X_{\text{selected}} = \{x_2, x_5, x_8, x_{12}, x_{17}\}$$

The **original** meaning of the selected features is preserved.

Why is feature selection required?

- Using **every available feature** does **not always** produce a **better model**.
- **Unnecessary** features can **introduce noise** and **increase model complexity**.

Advantages

- Reduces overfitting.
- Improves model generalization.
- Reduces model training time.
- Makes models easier to interpret.
- Reduces data-collection and storage costs.
- Helps address the curse of dimensionality.
- Can reduce multicollinearity.
- May improve prediction accuracy.

Feature
Study hours
Attendance
Previous marks
Sleep hours
Student roll number
Mobile Number

Example

Assume we predict house prices using:

- Area
 - Number of bedrooms
 - Location
 - Owner's phone number
 - Date on which data were entered
-
- The **owner's phone number** is **unlikely** to provide **meaningful information** about the house price.
 - It should normally be removed.

Types of unwanted features

Irrelevant features:

- An irrelevant feature has **little or no relationship** with the **target**.
- Example: Student **roll number** used to predict **examination performance**.

Redundant features:

- Two or more features may provide **nearly the same information**.

Example:

- Height in centimetres
- Height in metres

Keeping both does **not add meaningful information**.

Feature
Study hours
Attendance
Previous marks
Sleep hours
Student roll number
Mobile Number
Final marks

Noisy features

- These contain **errors**, **random variation**, or **unreliable** information that can confuse the model.

Constant features:

- A constant feature has the **same value** for every observation.

Student	Country code
A	91
B	91
C	91
D	91

Because the value does not change, the feature cannot distinguish between observations.

Near-constant features

These features have **almost the same value** for all observations and may provide very little predictive information.

Feature selection versus feature extraction

Feature selection	Feature extraction
Selects some existing features	Creates new features
Original meaning is preserved	Original meaning may be lost
Example: selecting age and income	Example: principal components
Easier to interpret	Usually less interpretable
• Methods include Recursive Feature Elimination (RFE) and Lasso	Methods include PCA and autoencoders

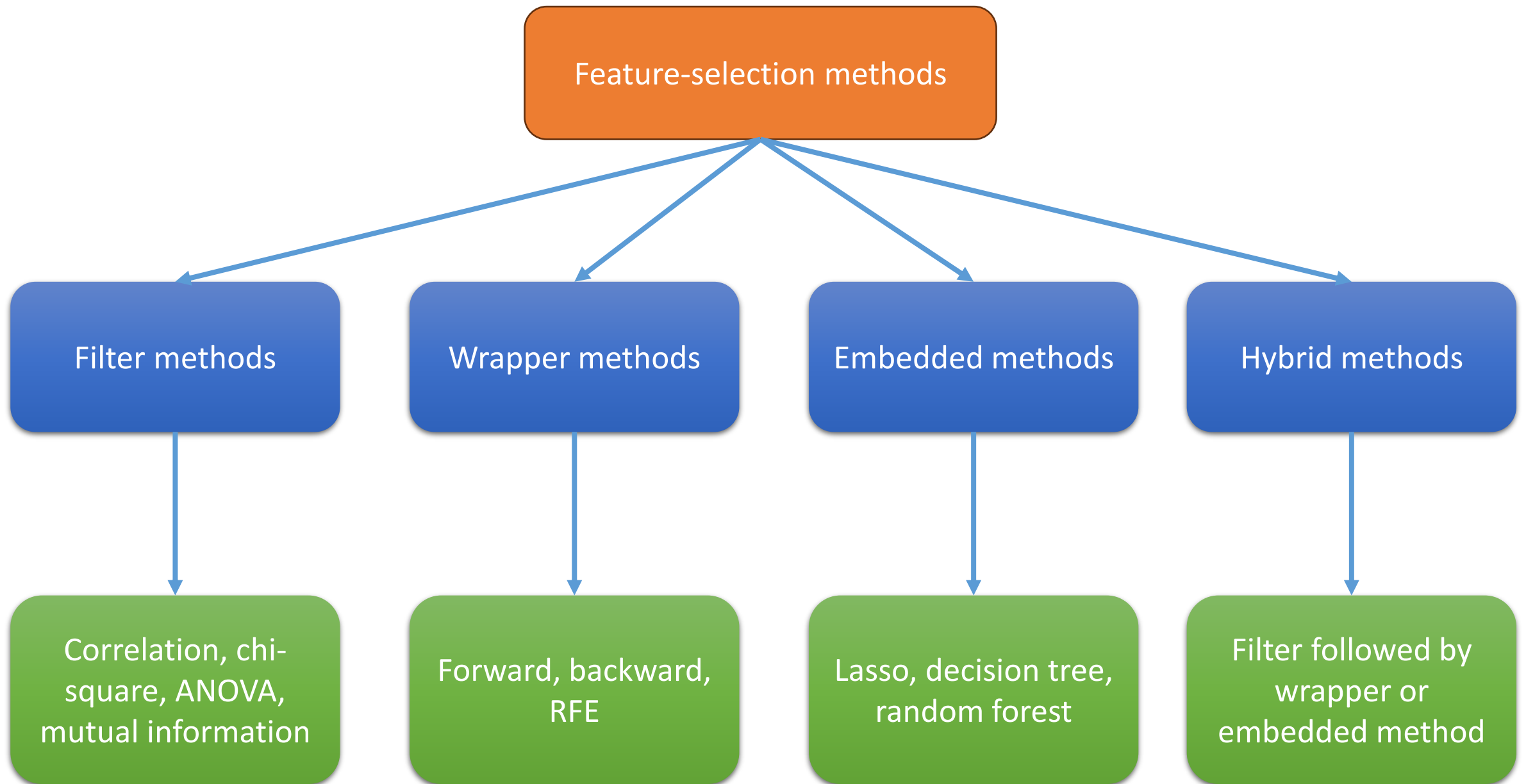
For example, if a dataset has ten features:

- Feature selection may retain **three original features**.
- PCA may **create three new features** from combinations of all ten features.

Categories of feature-selection methods

Feature-selection methods are broadly divided into:

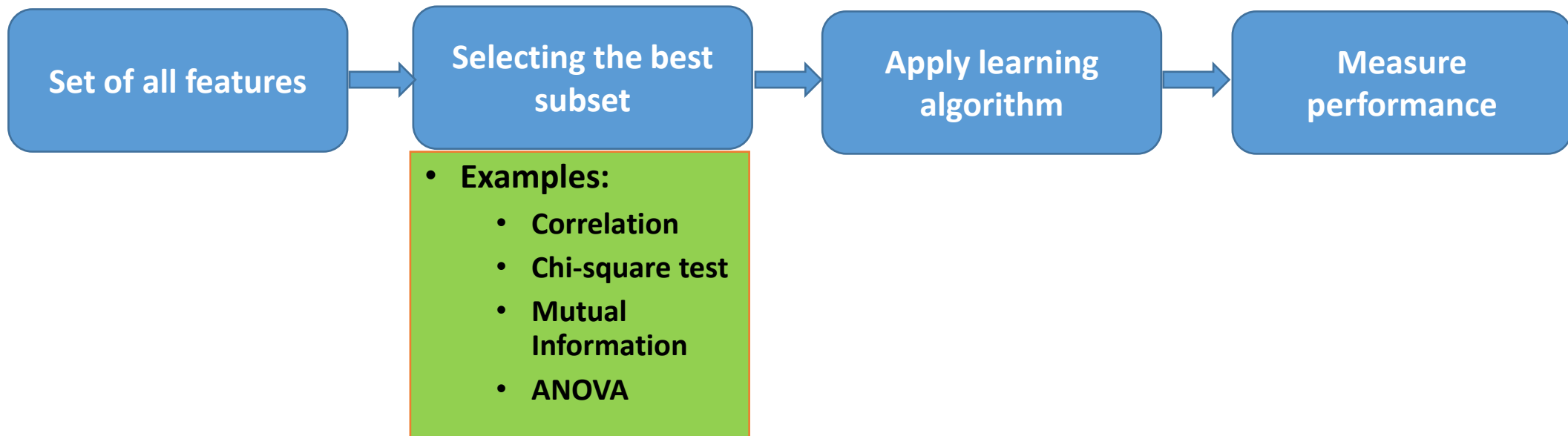
- Filter methods
- Wrapper methods
- Embedded methods
- Hybrid methods



Filter methods

- Filter methods are generally used as a preprocessing step.
- Filter methods **select features using statistical properties**, independently of a particular machine-learning model.

Filter Methods



General process

- Calculate a statistical score for every feature.
- Rank the features according to the score.
- Retain the highest-ranked features.
- Train the model using the selected features.

Advantages

- Fast and computationally efficient.
- Suitable for high-dimensional datasets.
- Independent of the machine-learning algorithm.
- Less likely to overfit than wrapper methods.

Limitations

- Often examine features individually.
- May not identify interactions between features.
- A feature that is weak individually may be useful when combined with another feature.

Variance threshold

- Features **with very little variation** generally **contain little useful** information.
- For a feature X , variance is:

$$\text{Var}(X) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

Sample	Feature A	Feature B
1	1	10
2	1	20
3	1	30
4	1	40

Feature A has zero variance and can be removed.

Limitation

A feature can have **high variance** and still be **unrelated to the target**. Therefore, variance alone does not measure predictive relevance.

Feature selection using scikit-learn

1. Dropping features which have low variance

1. Dropping features with zero variance
2. Dropping features with variance below the threshold variance

1. Dropping features with zero variance

- Variance threshold option drops two features with zero variance.
- Default variance threshold is zero.

`VarianceThreshold()`

`VarianceThreshold(threshold=0.0)`

offer	Age	online payment	items
1	35	0	5
1	26	0	4
1	41	0	9
1	34	0	10
1	38	0	3



Two features have zero variance

Dropping features with Zero variance

```
In [55]: from sklearn.feature_selection import VarianceThreshold  
selector = VarianceThreshold(threshold = 0.0)  
selector.fit_transform(dataset)
```

```
Out[55]: array([[35,  5],  
                [26,  4],  
                [41,  9],  
                [34, 10],  
                [38,  3]], dtype=int64)
```

offer	Age	online payment	items
1	35	0	5
1	26	0	4
1	41	0	9
1	34	0	10
1	38	0	3

Dropping features with low variance (less than threshold)

```
In [56]: # Importing the dataset
dataset = pd.read_csv('data-2.csv')
dataset
```

Out[56]:

	referred	repeat	Age	Promoted	items
0	1	0	35	1	5
1	0	0	26	0	4
2	1	0	41	1	9
3	1	1	34	0	10
4	1	0	38	1	3
5	1	0	40	0	7

Two features referred and repeat have low variance

Dropping features with low variance (less than threshold)

- If we want to drop a feature which contains only 0's 80% of the time or only 1s 80% of the time.
- Then the variance of the feature would be $0.8 * (1-0.8) = 0.16$

```
VarianceThreshold(threshold=0.16)
```

```
VarianceThreshold(threshold=(0.8 * (1-0.8)))
```

```
In [57]: from sklearn.feature_selection import VarianceThreshold  
selector = VarianceThreshold(threshold = 0.16)  
selector.fit_transform(dataset)
```

```
Out[57]: array([[35,  1,  5],  
                [26,  0,  4],  
                [41,  1,  9],  
                [34,  0, 10],  
                [38,  1,  3],  
                [40,  0,  7]], dtype=int64)
```

Two features referred and repeat have low variance.
Either only 1s or 0s for 80% of the time is dropped

Correlation-based selection

- Correlation measures the **strength of a linear relationship** between two **numerical** variables.
- The Pearson correlation coefficient is:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

- Its value lies between -1 and $+1$.

Value of r	Interpretation
Close to +1	Strong positive relationship
Close to -1	Strong negative relationship
Close to 0	Weak linear relationship

Correlation with the target

- For **predicting marks**, **MSE** marks may have a strong positive correlation with **final marks**.

Correlation among features

- If two input features are **highly correlated**, **one may be removed** to reduce redundancy and multicollinearity.

Example:

- Temperature in °C
- Temperature in °F

These two variables contain almost identical information.

Caution

- A **low Pearson correlation** does not necessarily mean that the **feature is useless**.
- The feature may have a strong **nonlinear** relationship with the target.

Chi-square test

- The chi-square test measures the association between a **feature** and a **categorical target**.
- It is commonly used for:
 - Categorical features
 - Non-negative count features
 - Text classification using word frequencies
- The statistic is:

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

where:

- O is the observed frequency.
- E is the expected frequency.
- A **larger chi-square** score indicates a **stronger association with the target**.

Example

- Suppose we study whether attendance category is associated with pass/fail status.

Attendance	Pass	Fail
High	18	2
Low	7	13

- Attendance appears to be related to the result.
- Scikit-learn's chi2 function requires **non-negative input values**.
- Standardized data containing **negative values should not be passed directly to it**.

ANOVA (Analysis of Variance) F-test

- ANOVA evaluates whether the mean of a numerical feature differs significantly across target classes.

- It is commonly used when:
 - Input feature: numerical
 - Target: categorical
 - The F-statistic is approximately:

$$F = \frac{\text{variance between groups}}{\text{variance within groups}}$$

- A **high F-score** suggests that the feature can distinguish between the classes.

Mutual information

- Mutual information measures how much knowledge of one variable reduces uncertainty about another variable.

$$I(X; Y) = \sum_{x,y} p(x, y) \log \left(\frac{p(x, y)}{p(x)p(y)} \right)$$

Important interpretation:

- $I(X; Y) = 0$: the variables are **independent**.
- **Larger** values: **greater** dependency.
- Mutual information can detect certain **nonlinear** relationships.

Suppose X represents attendance and Y represents examination result.

$$p(x, y) = P(X = x, Y = y)$$

Attendance X	Result Y	Number of students
High	Pass	40
High	Fail	10
Low	Pass	20
Low	Fail	30
Total		100

The joint probability of a student having **high attendance and passing** is:

$$p(X = \text{High}, Y = \text{Pass}) = \frac{40}{100} = 0.40$$

$$p(\text{High, Fail}) = \frac{10}{100} = 0.10$$

$$p(\text{Low, Pass}) = \frac{20}{100} = 0.20$$

$$p(\text{Low, Fail}) = \frac{30}{100} = 0.30$$

Attendance X	Result Y	Number of students
High	Pass	40
High	Fail	10
Low	Pass	20
Low	Fail	30
Total		100

The marginal probabilities $p(x)$ and $p(y)$ are calculated independently:

$$p(X = \text{High}) = \frac{40 + 10}{100} = 0.50$$

$$p(Y = \text{Pass}) = \frac{40 + 20}{100} = 0.60$$

Therefore, for the combination "High attendance and Pass," the ratio inside the logarithm is:

$$\frac{p(x, y)}{p(x)p(y)} = \frac{0.40}{(0.50)(0.60)} = \frac{0.40}{0.30} = 1.333$$

Here:

- $p(x, y) = 0.40$: actual probability of high attendance and passing together.
- $p(x)p(y) = 0.30$: expected probability if attendance and result were independent.

In simple terms, $p(x, y)$ tells us how frequently a particular feature value and target value occur together.

$$p(\text{High, Fail}) = \frac{10}{100} = 0.10$$

Attendance X	Result Y	Number of students
High	Pass	40
High	Fail	10
Low	Pass	20
Low	Fail	30
Total		100

The marginal probabilities $p(x)$ and $p(y)$ are calculated independently:

$$p(X = \text{High}) = \frac{40 + 10}{100} = 0.50$$

$$p(Y = \text{Fail}) = \frac{10 + 30}{100} = 0.40$$

Therefore, for the combination “High attendance and Fail,” the ratio inside the logarithm is:

$$\frac{p(x, y)}{p(x)p(y)} = \frac{0.1}{0.5 * 0.4} = 0.5$$

$p(x, y) = 0.1$: actual probability of high attendance and failing together

$p(x)p(y) = 0.2$: expected probability if attendance and result were independent

$p(x, y)$ tells us how frequently a particular feature value and target value occur together.

$$p(\text{Low}, \text{Pass}) = \frac{20}{100} = 0.20$$

Attendance X	Result Y	Number of students
High	Pass	40
High	Fail	10
Low	Pass	20
Low	Fail	30
Total		100

The marginal probabilities $p(x)$ and $p(y)$ are calculated independently:

$$p(X = \text{Low}) = \frac{20 + 30}{100} = 0.50$$

$$p(Y = \text{Pass}) = \frac{40 + 20}{100} = 0.60$$

Therefore, for the combination “Low attendance and Pass,” the ratio inside the logarithm is:

$$\frac{p(x, y)}{p(x)p(y)} = \frac{0.2}{0.5 * 0.6} = 0.67$$

$p(x, y) = 0.2$: actual probability of low attendance and passing together

$p(x)p(y) = 0.3$: expected probability if attendance and result were independent

$p(x, y)$ tells us how frequently a particular feature value and target value occur together.

$$p(\text{Low, Fail}) = \frac{30}{100} = 0.30$$

Attendance X	Result Y	Number of students
High	Pass	40
High	Fail	10
Low	Pass	20
Low	Fail	30
Total		100

The marginal probabilities $p(x)$ and $p(y)$ are calculated independently:

$$p(X = \text{Low}) = \frac{20 + 30}{100} = 0.50$$

$$p(Y = \text{Fail}) = \frac{10 + 30}{100} = 0.40$$

Therefore, for the combination “Low attendance and Pass,” the ratio inside the logarithm is:

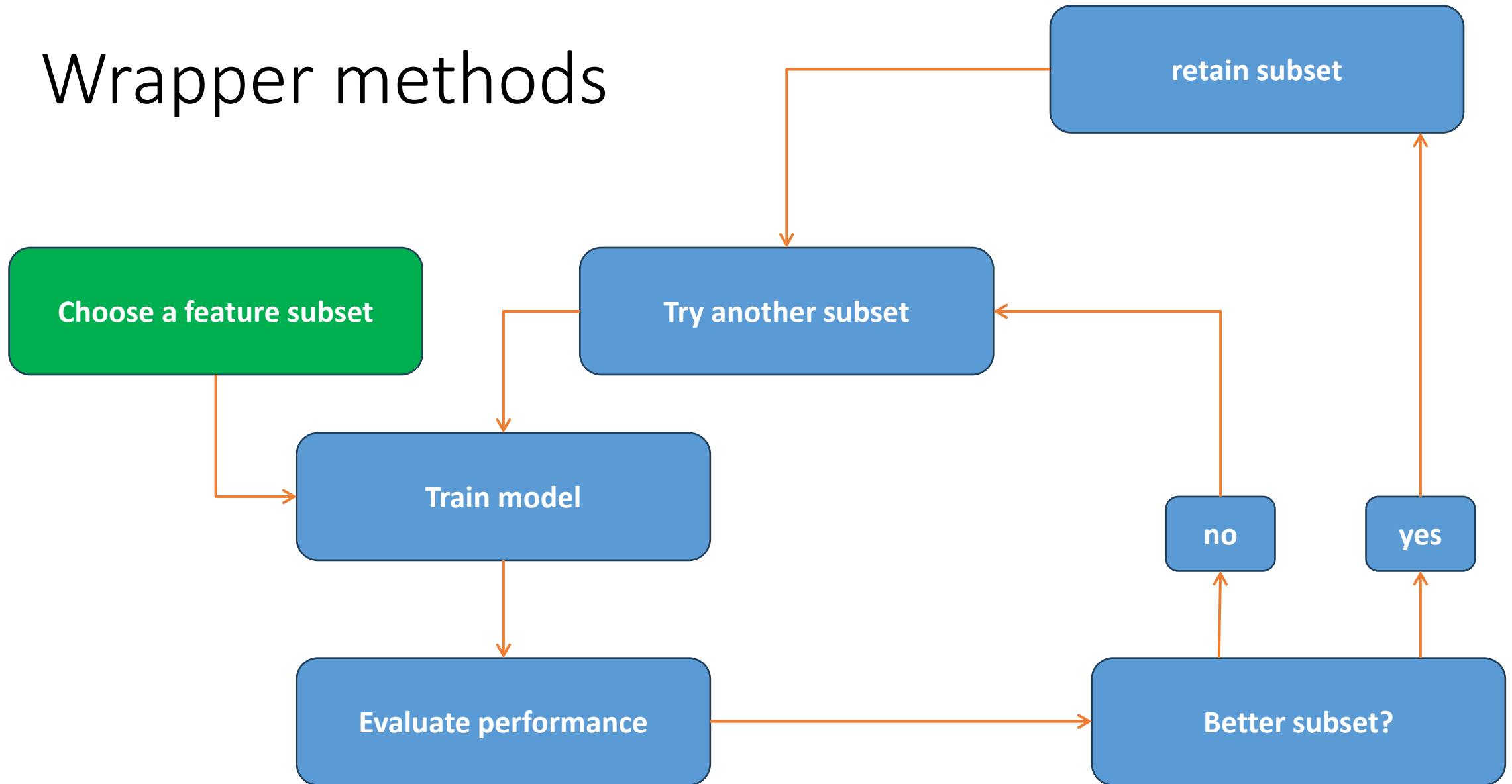
$$\frac{p(x, y)}{p(x)p(y)} = \frac{0.3}{0.5 * 0.4} = 1.5$$

$p(x, y) = 0.3$: actual probability of low attendance and passing together

$p(x)p(y) = 0.2$: expected probability if attendance and result were independent

$p(x, y)$ tells us how frequently a particular feature value and target value occur together.

Wrapper methods



Wrapper methods **evaluate different subsets** of features by **training and testing** a machine-learning model.

Wrapper methods

Advantages

- Consider the **relationship** between feature **subsets** and the **model**.
- Can identify useful feature **interactions**.
- Often achieve good **predictive** performance.

Limitations

- Computationally **expensive**.
- Can **overfit** small datasets.
- Results depend on the selected **model** and **evaluation** procedure.

Forward feature selection

- Forward selection begins without any features.

Procedure

- Start with an empty feature set.
- Test each feature individually.
- Add the feature that produces the **best performance**.
- Test the remaining features **together with the selected** features.
- Continue until the required number of features is selected.

Numerical illustration of forward selection

- Assume three available features:

X_1 : study hours

X_2 : attendance

X_3 : student ID

- The validation accuracies using individual features are:

Feature subset	Accuracy
X_1	82%
X_2	75%
X_3	51%

Feature subset	Accuracy
X_1	82%
X_2	75%
X_3	51%

- First, select X_1 , because it produces the highest accuracy.
- Next, test the remaining features with X_1 :

Feature subset	Accuracy
X_1, X_2	91%
X_1, X_3	81%

Select X_2 .

Finally:

Feature subset	Accuracy
X_1, X_2, X_3	89%

Adding X_3 reduces performance.
Therefore, the selected subset is:

$\{X_1, X_2\}$

Backward feature elimination

- Backward elimination begins with all features.

Procedure

- Train the model using every feature.
- Remove the least useful feature.
- Retrain and evaluate the model.
- Continue until the required number of features remains.

Example

Assume that we want to predict whether a student will **Pass or Fail** using the following four features:

- X_1 : Study hours
- X_2 : Attendance percentage
- X_3 : Previous examination marks
- X_4 : Student ID number

We use logistic regression and measure its performance using **5-fold cross-validation accuracy**.

Step 1: Begin with all features

The initial feature set is:

$$S_0 = \{X_1, X_2, X_3, X_4\}$$

Suppose the model produces:

Feature subset	Cross-validation accuracy
X_1, X_2, X_3, X_4	88%

The initial accuracy is:

Accuracy=88%

Step 2: Remove each feature separately

We temporarily remove one feature at a time and evaluate the model.

Removed feature	Remaining features	Accuracy
X_1	X_2, X_3, X_4	80%
X_2	X_1, X_3, X_4	84%
X_3	X_1, X_2, X_4	82%
X_4	X_1, X_2, X_3	91%

Removing X_4 , the student ID, produces the highest accuracy.

Therefore, remove X_4 permanently:

$$S_1 = \{X_1, X_2, X_3\}$$

The accuracy has increased from 88% to 91% because Student ID was an irrelevant or noisy feature.

Step 3: Test the remaining three features

The current feature set is:

$$S_1 = \{X_1, X_2, X_3\}$$

Again, remove each remaining feature temporarily.

Removed feature	Remaining features	Accuracy
X_1	X_2, X_3	83%
X_2	X_1, X_3	92%
X_3	X_1, X_2	85%

Removing attendance X_2 gives the highest accuracy: $S_2 = \{X_1, X_3\}$

The accuracy improves slightly: 91% $\rightarrow$ 92%

This may mean that attendance provides information already represented by **study hours and previous marks**.

Step 4: Test the remaining two features

The current feature set is:

$$S_2 = \{X_1, X_3\}$$

Test the removal of each feature:

Removed feature	Remaining feature	Accuracy
X_1	X_3	84%
X_3	X_1	81%

Both removals reduce accuracy substantially.

- Removing X_1 reduces accuracy from 92% to 84%.
- Removing X_3 reduces accuracy from 92% to 81%.

Therefore, the elimination process stops.

Final selected features

The final feature subset is:

$$S_{\text{final}} = \{X_1, X_3\}$$

- Study hours
- Previous examination marks

The final model accuracy is:

92%

Iteration	Available features	Removed feature	Accuracy after removal
Initial	X_1, X_2, X_3, X_4	None	88%
1	X_1, X_2, X_3	X_4	91%
2	X_1, X_3	X_2	92%
Stop	X_1, X_3	None	92%

It is generally **more expensive** than forward selection when the dataset contains **many features**.

Recursive Feature Elimination

- Recursive Feature Elimination, or **RFE**, repeatedly trains a model and removes the least important feature or features.

Procedure

- Train a model with all features.
- Obtain feature coefficients or importance scores.
- Remove the least important feature.
- Retrain the model.
- Repeat until the desired number of features remains.

Example

- Suppose we want to predict whether a student will **Pass or Fail** using four features:

Feature	Description
X_1	Study hours
X_2	Attendance
X_3	Previous marks
X_4	Student ID

Our goal is to select the best **two features**.

Assume we use **logistic regression** as the estimator.

The **absolute coefficient magnitude** represents the importance of each feature.

Iteration 1: Train using all four features

The logistic-regression model is: $z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_4 X_4$

Suppose the trained model produces these coefficients:

Feature	Coefficient	Absolute coefficient
X_1 : Study hours	1.20	1.20
X_2 : Attendance	0.45	0.45
X_3 : Previous marks	1.50	1.50
X_4 : Student ID	0.05	0.05

RFE considers the absolute coefficient magnitude: $|\beta_j|$

The smallest value is: $|\beta_4| = 0.05$

Therefore, RFE removes X_4 , Student ID. The remaining subset is: $\{X_1, X_2, X_3\}$

Iteration 2: Retrain using three features

The model is retrained after removing X_4 .

The coefficients may change because the model has been refitted.

Suppose the new coefficients are:

Feature	New coefficient	Absolute coefficient
X_1 : Study hours	1.10	1.10
X_2 : Attendance	0.30	0.30
X_3 : Previous marks	1.65	1.65

The smallest absolute coefficient is: $|\beta_2| = 0.30$

Therefore, RFE removes X_2 , Attendance.

The remaining subset is: $\{X_1, X_3\}$

Stopping condition

We requested two features, and two features now remain. Therefore, RFE stops.
The selected features are:

$X_1 = \text{Study hours}$

$X_3 = \text{Previous marks}$

Iteration	Features used	Least important feature		Action
1	X_1, X_2, X_3, X_4	X_4		Remove X_4
2	X_1, X_2, X_3	X_2		Remove X_2
Final	X_1, X_3	—		Stop

The elimination sequence is: $\{X_1, X_2, X_3, X_4\} \rightarrow \{X_1, X_2, X_3\} \rightarrow \{X_1, X_3\}$

Recursive Feature Elimination with CV (RFECV)

- RFECV combines Recursive Feature Elimination with cross-validation.
- It automatically identifies an appropriate number of features.
- But RFECV uses **cross-validation** to decide whether the model should retain **one, two, three, or more features**.

RFECV combines:

- Recursive Feature Elimination
- Cross-validation
- Model-performance evaluation

How RFECV works

RFECV follows these steps:

- Begin with all features.
- Train the estimator.
- Rank features using coefficients or feature importance.
- Remove the least important feature.
- Evaluate the remaining subset using cross-validation.
- Repeat the elimination process.
- Select the number of features producing the best cross-validation score.
- Refit the estimator using the selected features.

Example

- Suppose we want to predict whether a student will pass or fail using four features:

Feature	Description
X_1	Study hours
X_2	Attendance percentage
X_3	Previous marks
X_4	Student ID

We use:

- Logistic regression
- 5-fold cross-validation
- Accuracy as the evaluation measure
- Step = 1, meaning one feature is removed at each iteration

Iteration 1: All four features

The initial feature set is:

$$S_4 = \{X_1, X_2, X_3, X_4\}$$

Suppose logistic regression produces the following absolute coefficients:

Feature	Absolute coefficient
X_1 : Study hours	1.20
X_2 : Attendance	0.45
X_3 : Previous marks	1.50
X_4 : Student ID	0.05

Because X_4 has the smallest absolute coefficient, RFE removes it.

The 5-fold accuracies for the four-feature model are:

Fold	Accuracy
1	88%
2	86%
3	90%
4	87%
5	89%

The mean cross-validation accuracy is:

$$\frac{88 + 86 + 90 + 87 + 89}{5} = 88\%$$

Iteration 2: Three features

After removing X_4 , the remaining subset is:

$$S_3 = \{X_1, X_2, X_3\}$$

After retraining, assume the coefficients are:

Feature	Absolute coefficient
X_1 : Study hours	1.10
X_2 : Attendance	0.30
X_3 : Previous marks	1.65

Therefore, X_2 is removed next.

The cross-validation accuracies are:

Fold	Accuracy
1	90%
2	91%
3	89%
4	92%
5	88%

The mean is:

$$\frac{90 + 91 + 89 + 92 + 88}{5} = 90\%$$

Iteration 3: Two features

The remaining subset is:

$$S_2 = \{X_1, X_3\}$$

Suppose the model produces:

Feature	Absolute coefficient
X_1 : Study hours	1.25
X_3 : Previous marks	1.70

Of the remaining features, X_1 has the smaller coefficient and is removed to evaluate the one-feature model.

The two-feature model produces:

Fold	Accuracy
1	92%
2	94%
3	91%
4	93%
5	90%

The mean accuracy is:

$$\frac{92 + 94 + 91 + 93 + 90}{5} = 92\%$$

Iteration 4: One feature

The remaining feature is:

$$S_1 = \{X_3\}$$

The cross-validation accuracies are:

Fold	Accuracy
1	84%
2	86%
3	85%
4	83%
5	87%

The mean accuracy is:

$$\frac{84 + 86 + 85 + 83 + 87}{5} = 85\%$$

Selecting the best subset

RFECV compares the average cross-validation scores:

Number of features	Selected subset on elimination path	Mean CV accuracy
4	X_1, X_2, X_3, X_4	88%
3	X_1, X_2, X_3	90%
2	X_1, X_3	92%
1	X_3	85%

The highest cross-validation accuracy is obtained using two features:

Best mean CV accuracy = 92%

Therefore, RFECV selects:

X_1 = Study hours

X_3 = Previous marks

The elimination path is:

$$\{X_1, X_2, X_3, X_4\} \rightarrow \{X_1, X_2, X_3\} \rightarrow \{X_1, X_3\} \rightarrow \{X_3\}$$

RFECV evaluates every subset along this elimination path and returns the subset with the highest cross-validation score.

Embedded methods

- Embedded methods perform feature selection **as part of model training**.
- They provide a balance between the **speed of filter methods** and the predictive **ability of wrapper methods**.

Common embedded methods include:

- Lasso regression
- L1-regularized logistic regression
- Decision trees
- Random forests
- Gradient-boosted trees

Lasso regression

- Lasso adds an L1 penalty to the ordinary regression objective:

$$\text{Objective} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

where:

- β_j is the coefficient of feature j .
- λ , represented by alpha in Scikit-learn, controls the penalty strength.
- Lasso can **force some coefficients to exactly zero**.
- Features with zero coefficients are effectively **removed**.

Effect of λ

- Small λ : weak penalty and more selected features.
- Large λ : strong penalty and fewer selected features.
- Excessively large λ : important features may also be removed.

Why scaling is important

- Lasso penalizes coefficient magnitudes.
- If features have different scales, the penalty may treat them unfairly.
- Therefore, numerical features should generally be standardized before applying Lasso.

L1-regularized logistic regression

- For classification, L1-regularized logistic regression can eliminate features by setting their coefficients to zero.

Decision-tree feature importance

- Decision trees select features while creating splits.
- A feature becomes important when it produces a large reduction in impurity.
- For classification, common impurity measures include:

$$\text{Entropy (S)} \equiv -(p_{\oplus} \log_2 p_{\oplus} + p_{\ominus} \log_2 p_{\ominus})$$

$$Gini = 1 - \sum_{i=1}^C (p_i)^2$$

- Impurity-based importance can be **biased toward continuous features** or variables with **many possible split points**.
- Permutation importance can provide a useful complementary analysis.

Permutation importance

- Permutation importance measures the **reduction in model performance** after randomly shuffling one feature.

Basic idea

- Calculate the **original** model performance.
- **Shuffle** one feature.
- Calculate the new performance.
- Measure the **decrease**.

$$\text{Importance}_j = \text{Original score} - \text{Score after shuffling feature } j$$

- A **large decrease** means that the **model relies strongly on that feature**.
- Permutation importance should preferably be calculated using unseen **validation or test** data.

Comparison of feature-selection methods

Method	Category	Model required?	Speed	Detects interactions?	Common use
Variance threshold	Filter	No	Very fast	No	Remove constant features
Correlation	Filter	No	Fast	No	Numerical regression data
Chi-square	Filter	No	Fast	No	Classification and count data
ANOVA F-test	Filter	No	Fast	No	Numerical features and class target
Mutual information	Filter	No	Moderate	Limited	Nonlinear dependency
Forward selection	Wrapper	Yes	Slow	Yes	Small feature sets
Backward elimination	Wrapper	Yes	Slow	Yes	Small or moderate feature sets
RFE	Wrapper	Yes	Slow	Partly	Model-specific selection
Lasso	Embedded	Yes	Moderate	Limited	Regression
L1 logistic regression	Embedded	Yes	Moderate	Limited	Classification
Tree importance	Embedded	Yes	Moderate	Yes	Nonlinear problems
Permutation importance	Model-based	Yes	Moderate	Reflects fitted model	Model interpretation

Choosing a method based on data types

Input feature	Target	Suitable method
Numerical	Numerical	Correlation, f_regression, mutual information
Numerical	Categorical	ANOVA, mutual information
Categorical/count	Categorical	Chi-square, mutual information
Many numerical features	Numerical	Lasso
Many features	Categorical	L1 logistic regression
Mixed/nonlinear data	Numerical or categorical	Tree-based importance
Small number of features	Any	Wrapper methods
Very high-dimensional data	Any	Filter method followed by an embedded method

Categorical variables generally need to be encoded before being passed to Scikit-learn estimators.

Correct feature-selection workflow

One of the most important rules is:

- **Split the data** before learning which features should be selected.
- If **feature selection** is performed using the **entire dataset**, **information from the test set** influences the **selected features**.
- This is called **data leakage**.
- The pipeline ensures that **feature selection** is **learned only from the training data during cross-validation**.

Selecting the best number of features

- The value of k in SelectKBest should **not** normally be selected by looking at **test-set performance**.
- It can be chosen using **cross-validation**.

Common mistakes

Selecting features before splitting the dataset

- This causes data leakage.

Using the test set repeatedly

- The test set should be used only for final evaluation. Use **cross-validation** on the training data **to select features** and **hyperparameters**.

Removing features only because correlation is low

- Pearson correlation detects linear relationships.
- A low correlation may hide a **useful nonlinear relationship**.

Applying chi-square to negative values

- Scikit-learn's chi-square selector requires **non-negative** input features.

Ignoring feature scaling for Lasso

- Because Lasso penalizes coefficients, features should generally be **standardized**.

Selecting features only from domain knowledge

- Domain knowledge is valuable, but it should be **combined with statistical** and predictive evidence.

Treating feature importance as causality

- A feature may **help prediction without being the cause of the target.**

Removing one of two correlated features without checking context

- Two correlated features may represent different real-world measurements and **may behave differently** under changing operating conditions.

Recommended practical strategy

For a dataset with many features:

- Remove identifiers, duplicate columns, and obvious leakage variables.
- Remove constant or near-constant features.
- Inspect missing values and data quality.
- Examine correlation and redundancy.
- Apply a suitable filter method.
- Apply an embedded or wrapper method to the reduced dataset.
- Select hyperparameters through cross-validation.
- Evaluate the final pipeline once on the test set.
- Compare performance, complexity, stability, and interpretability.

- A hybrid procedure is often effective:

Original features → Filter method → Embedded/wrapper method → Final model

Univariate feature selection

- The Iris Dataset contains four features
 - (length and width of sepals and petals) of 50 samples of three species of Iris (Iris setosa, Iris virginica and Iris versicolor).
 - These measures were used to create a linear discriminant model to classify the species.



Iris Versicolor



Iris Setosa



Iris Virginica

Iris Setosa:

Length of sepals: 5.1

Width of sepals: 3.5

Length of petals: 1.4

Width of petals : 0.2

5.1,3.5,1.4,0.2,Iris-setosa

Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
1	5.1	3.5	1.4	0.2	Iris-setosa
			:		
25	4.9	3	1.4	0.2	Iris-versicolor
			:		
39	4.7	3.2	1.3	0.2	Iris-virginica

Univariate Feature Selection

```
In [7]: from sklearn.feature_selection import SelectKBest  
        from sklearn.feature_selection import chi2  
        from sklearn.datasets import load_iris
```

```
In [8]: iris = load_iris()  
        X, y = iris.data, iris.target  
        X.shape
```

```
Out[8]: (150, 4)
```

Original dataset
contains 4
predictors

```
In [9]: #Select 3 best features  
        X_select = SelectKBest(chi2, k=3).fit_transform(X, y)  
        X_select.shape
```

```
Out[9]: (150, 3)
```

Best 3 predictors
are retained based
on chi-square
value

Model Based Feature Selection

```
In [62]: from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.feature_selection import SelectFromModel
iris = load_iris()
X, y = iris.data, iris.target
X.shape
```

Estimate feature importance using **RandomForest** model to select features.

```
Out[62]: (150, 4)
```

```
In [64]: clf = RandomForestClassifier(n_estimators=10)
clf = clf.fit(X, y)
clf.feature_importances_
```

n_estimators – number of decision trees in the forest.

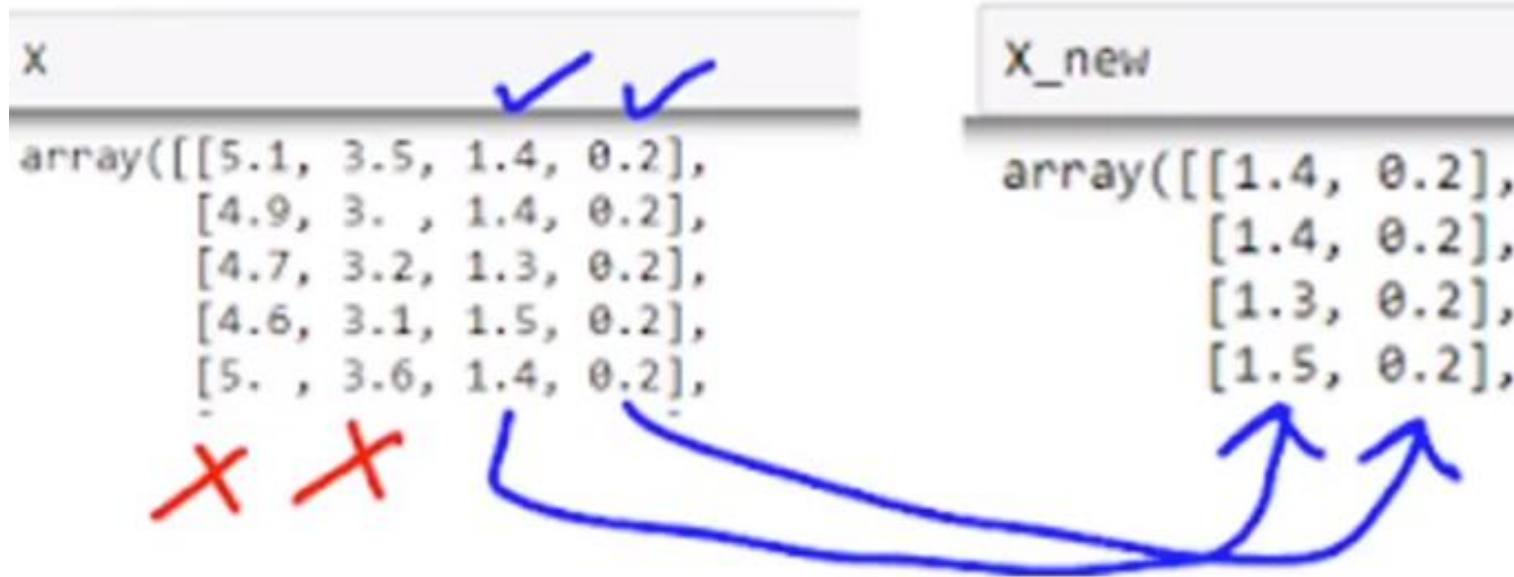
```
Out[64]: array([0.24504133, 0.04692402, 0.39381251, 0.31422214])
```

```
model = SelectFromModel(clf, prefit = True)
```

```
In [65]: model = SelectFromModel(clf, prefit = True)
X_new = model.transform(X)
X_new.shape
```

Out[65]: (150, 2)

Out of 4 only 2 features
have been retained



In [17]: X

```
Out[17]: array([[5.1, 3.5, 1.4, 0.2],
                [4.9, 3. , 1.4, 0.2],
                [4.7, 3.2, 1.3, 0.2],
                [4.6, 3.1, 1.5, 0.2],
                [5. , 3.6, 1.4, 0.2],
                [5.4, 3.9, 1.7, 0.4],
                [4.6, 3.4, 1.4, 0.3],
                [5. , 3.4, 1.5, 0.2],
                [4.4, 2.9, 1.4, 0.2],
                [4.9, 3.1, 1.5, 0.1],
                [5.4, 3.7, 1.5, 0.2],
                [4.8, 3.4, 1.6, 0.2],
                [4.8, 3. , 1.4, 0.1],
                [4.3, 3. , 1.1, 0.1],
                [5.8, 4. , 1.2, 0.2],
                [5.7, 4.4, 1.5, 0.4],
                [5.4, 3.9, 1.3, 0.4],
                [5.1, 3.5, 1.4, 0.3],
```

In [18]: X_new

```
Out[18]: array([[1.4, 0.2],
                [1.4, 0.2],
                [1.3, 0.2],
                [1.5, 0.2],
                [1.4, 0.2],
                [1.7, 0.4],
                [1.4, 0.3],
                [1.5, 0.2],
                [1.4, 0.2],
                [1.5, 0.1],
                [1.5, 0.2],
                [1.6, 0.2],
                [1.4, 0.1],
                [1.1, 0.1],
                [1.2, 0.2],
                [1.5, 0.4],
                [1.3, 0.4],
```

Feature Selection Using Pipeline

```
In [ ]: from sklearn.neighbors import KNeighborsClassifier
        from sklearn.pipeline import Pipeline
        clfs = Pipeline([('feature_selection',
                           SelectFromModel(RandomForestClassifier(n_estimators=10))),
                           ('classification', KNeighborsClassifier())])
        model = clfs.fit(X, y)
```

Pipeline process:

1. First features are selected
2. Model is built using selected features

Thank you